Employee Turnover Data

Data Profiling and Cleaning Report

Earl Birdsong | Python 3.12, pandas, NumPy, Matplotlib, seaborn

Executive Summary

A technology firm wants to understand which employees are at greater risk of leaving so it can design better retention strategies. Before any modeling, the raw HR dataset had to be profiled and cleaned. This report documents the dataset's structure, the quality issues found, how each was corrected in Python, and the trade-offs of that approach.

Result: the raw file of 10,199 records was reduced to 10,100 unique records with all 16 variables retained, no missing values, consistent category labels, numeric fields stored as numbers, and implausible values corrected.

1. Data Profile

The raw dataset contains **10,199 records (rows)** and **16 variables (columns)**. Table 1 lists each variable with its data type, subtype, and a sample of observed raw values.

Table 1. *Variable profile*

Variable	Data type	Subtype	Sample values (raw)
EmployeeNumber	Qualitative	Nominal	1, 2, 3, 4
Age	Quantitative	Discrete	28, 33, 22, 23
Tenure	Quantitative	Discrete	6, 2, 1, 16
Turnover	Qualitative	Nominal	Yes, No
HourlyRate	Quantitative	Continuous	$24.37, $22.52, $88.77
HoursWeekly	Quantitative	Discrete	40
CompensationType	Qualitative	Nominal	Salary
AnnualSalary	Quantitative	Continuous	50689.6, 46841.6, 284641.6
DrivingCommuterDistance	Quantitative	Continuous	89, 35, 12, 0
JobRoleArea	Qualitative	Nominal	Research, Sales, Information_Technology
Gender	Qualitative	Nominal	Female, Male, Prefer Not to Answer
MaritalStatus	Qualitative	Nominal	Married, Single, Divorced
NumCompaniesPreviouslyWorked	Quantitative	Discrete	3, 6, 1, 7
AnnualProfessionalDevHrs	Quantitative	Discrete	7, 8, 19, 23
PaycheckMethod	Qualitative	Nominal	Mail Check, Mailed Check, DirectDeposit
TextMessageOptIn	Qualitative	Nominal	Yes, No

EmployeeNumber is stored as an integer but is classified as nominal because it is an identifier with no numeric meaning. HoursWeekly and CompensationType each hold a single value for every employee; they were kept to preserve the complete variable list.

2. Data Quality Inspection and Findings

Each issue was checked in a single reproducible Python script using pandas and NumPy, with boxplots drawn in Matplotlib and seaborn. Table 2 summarizes the inspection method and what it found.

Table 2. *Quality issues: detection and findings*

Issue	How it was inspected	Findings
Duplicate entries	df.duplicated().sum() on full rows; EmployeeNumber checked separately for repeated IDs	99 exact duplicate rows, each repeating an existing EmployeeNumber
Missing values	df.isna().sum() with percentage of rows per column	NumCompaniesPreviouslyWorked: 665 (6.5%); AnnualProfessionalDevHrs: 1,969 (19.3%); TextMessageOptIn: 2,266 (22.2%)
Inconsistent entries	value_counts() on each categorical column; AnnualSalary compared with HourlyRate × HoursWeekly × 52	JobRoleArea: 228 rows with variant spellings (e.g., Human_Resources). PaycheckMethod: 7 labels for 2 methods. AnnualSalary: 2,149 rows disagree with the hourly rate, including 57 negative salaries
Formatting errors	repr() on column names; dtypes and head() to confirm storage types	HourlyRate header has a trailing space; values are text with a $ sign and trailing space; count columns stored as decimals
Outliers	describe() summary statistics, IQR fences (Q1 − 1.5×IQR, Q3 + 1.5×IQR, and boxplots	DrivingCommuterDistance: 1,351 negative values and extreme values of 250–950 miles. Age, Tenure, and the count columns had no outliers

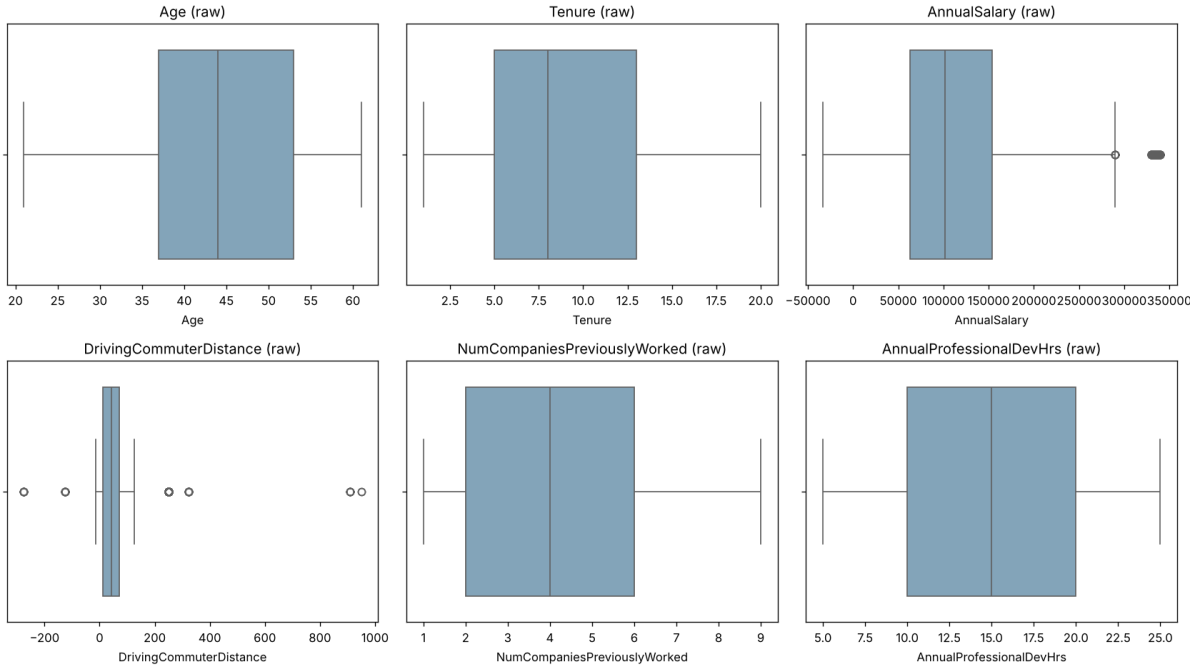


Figure 1. *Boxplots of the raw numeric variables. AnnualSalary shows negative and inflated values; DrivingCommuterDistance shows negative and extreme values.*

3. Data Cleaning

Modifications and Rationale

Table 3 describes how the dataset was modified for each issue and why that technique was chosen.

Table 3. *Cleaning techniques applied*

Issue	Modification in Python	Why this technique
Duplicates	drop_duplicates(keep="first")	Rows were exact copies, so removing them loses no information and prevents double-counting employees
Missing values	Median imputation for the two numeric counts; "Unknown" for TextMessageOptIn	The median resists skew and keeps whole-number counts. Labeling opt-in as Unknown avoids inventing a Yes/No answer
Inconsistent entries	Mapping dictionaries for JobRoleArea and PaycheckMethod; AnnualSalary recalculated as HourlyRate × 40 × 52	Mapping merges variants into one category per value. HoursWeekly is constant, so the hourly rate is the reliable source for salary
Formatting errors	str.strip() on headers and text; regex removal of $ and spaces; to_numeric() and astype(int)	Numeric columns must be numeric for calculation and modeling; whole-number counts are stored as integers
Outliers	abs() on negative commutes; values beyond the IQR fence set to NaN and replaced with the median (41 miles)	Small negatives are sign-entry errors. Commutes of 250+ miles are not plausible daily trips, and median replacement keeps the records

After cleaning, the script verifies zero missing values, zero duplicates, and no negative salaries or commutes, then exports the result to employee_turnover_cleaned.csv.

Advantages

- **Every employee is retained.** Imputing and replacing values instead of deleting rows keeps all 10,100 unique records, which preserves statistical power for later turnover modeling.
- **Reproducible and auditable.** All changes run from one annotated script against an untouched raw file, so the same cleaned output can be regenerated and every rule reviewed.

Limitations

- **Imputation reduces variability.** Filling 19% of AnnualProfessionalDevHrs and the replaced commute outliers with medians concentrates values at the center, which can weaken relationships a model might detect.
- **Some corrections rely on assumptions.** Treating negative commutes as sign errors and recalculating salary from the hourly rate assume the hourly rate is correct. TextMessageOptIn also now carries an Unknown category that later analysis must handle.